Lista de Chequeo Actividad 7

Criterios	Calificaciones	Pts
Este criterio depende de una competencia de aprendizaje Carga del dataset y tokenización por palabras de cada una de las sentencias. La idea es la de obtener una lista de listas de palabras tokenizadas del dataset.	**0.25 pts100%** **0.2 pts80%** **0.15 pts60%** **0.1 pts40%** **0.05 pts20%** **0 pts0%**	0.25 pts
Este criterio depende de una competencia de aprendizaje Entrenamiento de los modelos Word2vec y FastText Proyecto 1: 10%	**0.5 pts100%** **0.4 pts80%** **0.3 pts60%** **0.2 pts40%** **0.1 pts20%** **0 pts0%**	0.5 pts
Este criterio depende de una competencia de aprendizaje Consulta de similitud semántica. La idea es que sea implementada una función para que dada una palabra, ésta devuelva las 10 palabras más similares en los dos modelos.	**0.25 pts100%** **0.2 pts80%** **0.15 pts60%** **0.1 pts40%** **0.05 pts20%** **0 pts0**	0.25 pts

Este criterio depende de una competencia de aprendizaje Vizualizar en el plano cartesiano los vectores embebidos del dataset para los dos modelos. Números de sentencias de 50, 100 y 300 y embeddings de tamaño 300	**0.5 pts100%** **0.4 pts80%** **0.3 pts60%** **0.2 pts40%** **0.1 pts20%** **0 pts0%**	0.5 pts
Este criterio depende de una competencia de aprendizaje Cargar los conjuntos de train, validación y testeo del script de Biobert. Proyecto 1: 10%	**0.5 pts100%** **0.4 pts80%** **0.3 pts60%** **0.2 pts40%** **0.1 pts20%** **0 pts0%**	0.5 pts
Este criterio depende de una competencia de aprendizaje Imprimir las sentencias [0] de validación y train con sus respectivas etiquetas. La idea es mostrar La sentencia train[0] con sus etiquetas tag[0] y la sentencia validation[0] con su etiqueta tag[0].	**0.5 pts100%** **0.4 pts80%** **0.3 pts60%** **0.2 pts40%** **0.1 pts20%** **0 pts0%**	0.5 pts

Este criterio depende de una competencia de aprendizaje Tokenización de las sentencias [0] del train y valadación. Usando el tokenizador de BETO (dccuchile/bert-base-spanish-wwm-cased)	**0.5 pts100%** **0.4 pts80%** **0.3 pts60%** **0.2 pts40%** **0.1 pts20%** **0 pts0%**	0.5 pts
Este criterio depende de una competencia de aprendizaje Carga y procesamiento del conjunto de pdfs para la tarea de embeddings de sentencias. Usar Pymupdf4llm	**0.5 pts100%** **0.4 pts80%** **0.3 pts60%** **0.2 pts40%** **0.1 pts20%** **0 pts0**	0.5 pts
Este criterio depende de una competencia de aprendizaje Fragmentar el texto usando la técnica de RecursiveCharactersplitter para obtener los chunks. Chunk_size=1000 chunk_overlap=200	**0.25 pts100%** **0.2 pts80%** **0.15 pts60%** **0.1 pts40%** **0.05 pts20%** **0 pts0%**	0.25 pts
Este criterio depende de una competencia de aprendizaje Generar los embeddings de los chunks usando los modelos de sentence-transformers.	**0.5 pts100%** **0.4 pts80%** **0.3 pts60%** **0.2 pts40%** **0.1 pts20%** **0 pts0%**	0.5 pts

Este criterio depende de una competencia de aprendizaje Dada una sentencia de entrada obtener su representación vectorial y la sentencia más similar para cada uno de los modelos. La idea es que la sentencia de entrada sea vectorizada y se determine la sentencia más similar para cada uno de los 4 model.	**0.5 pts100%** **0.4 pts80%** **0.3 pts60%** **0.2 pts40%** **0.1 pts20%** **0 pts0%**	0.5 pts
Este criterio depende de una competencia de aprendizaje Vizualizar en el plano cartesiano la sentencia de entrada con la sentencia más similar para cada modelo. Usar PCA.	**0.25 pts100%** **0.2 pts80%** **0.15 pts60%** **0.1 pts40%** **0.05 pts20%** **0 pts0%**	0.25 pts